

```
[ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

...

```
[ ]: # @title Default title text
from google.colab import files
uploaded = files.upload()
```

No file chosen, Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving house_data.csv to house_data (1).csv

```
[ ]: import pandas as pd

df = pd.read_csv("house_data.csv")
df.head()
```

[20]:

```
[20]:
```

```
.....
bhk      type      locality area  price  price_unit      region      status  age
0    3  Apartment  Lak And Hanware The Residency Tower  685    2.50      Cr  Andheri West  Ready to move  New
1    2  Apartment      Radheya Sai Enclave Building No 2  640   52.51      L  Naigaon East  Under Construction  New
2    2  Apartment                        Romell Serene  610    1.73      Cr  Borivali West  Under Construction  New
3    2  Apartment  Soundlines Codename Urban Rainforest  876   59.98      L      Panvel  Under Construction  New
4    2  Apartment                        Origin Oriana  659   94.11      L  Mira Road East  Under Construction  New
```

```
[ ]: import io

filename = list(uploaded.keys())[0] # first uploaded file
df = pd.read_csv("house_data.csv")

print("Dataset Loaded Successfully ✅")
df.head()

Dataset Loaded Successfully ✅
```

[21]:

```
.....
bhk      type      locality area  price  price_unit      region      status  age
0    3  Apartment  Lak And Hanware The Residency Tower  685    2.50      Cr  Andheri West  Ready to move  New
1    2  Apartment      Radheya Sai Enclave Building No 2  640   52.51      L  Naigaon East  Under Construction  New
2    2  Apartment                        Romell Serene  610    1.73      Cr  Borivali West  Under Construction  New
3    2  Apartment  Soundlines Codename Urban Rainforest  876   59.98      L      Panvel  Under Construction  New
4    2  Apartment                        Origin Oriana  659   94.11      L  Mira Road East  Under Construction  New
```

```
'''  
[ ]: print("Columns in Dataset:")  
print(df.columns)
```

```
Columns in Dataset:  
,Index(['bhk', 'type', 'locality', 'area', 'price', 'price_unit', 'region',  
,      'status', 'age'],  
,      dtype='object')
```

```
[ ]: # Change these column names according to your CSV  
X = df[['area', 'locality', 'bhk']] # Features  
y = df['price']                    # Target
```

```
[ ]: import pandas as pd  
import numpy as np  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score  
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import OneHotEncoder  
from sklearn.pipeline import Pipeline  
  
# Assuming X and y are already defined from previous cells (X = df[['area', 'locality', 'bhk']], y = df['price'])  
  
# Identify categorical and numerical columns from the selected features X  
categorical_cols = X.select_dtypes(include=['object']).columns  
numerical_cols = X.select_dtypes(exclude=['object']).columns  
  
# Preprocessing pipeline for categorical and numerical features  
preprocessor = ColumnTransformer(  
    transformers=[  
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols),  
        ('num', 'passthrough', numerical_cols)  
    ]  
)  
  
# Create a full pipeline with preprocessing and the linear regression model  
model = Pipeline(steps=[  
    ('preprocessor', preprocessor),  
    ('regressor', LinearRegression())  
)  
  
# Split data into training and testing sets  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
# Train the model  
model.fit(X_train, y_train)  
  
# Make predictions  
y_pred = model.predict(X_test)  
  
# Evaluate the model  
print("----- Model Performance -----")  
print("MAE :", mean_absolute_error(y_test, y_pred))  
print("MSE :", mean_squared_error(y_test, y_pred))  
print("RMSE :", np.sqrt(mean_squared_error(y_test, y_pred)))  
print("R2 :", r2_score(y_test, y_pred))
```

```
----- Model Performance -----  
,MAE : 15.264000699647285  
,MSE : 511.0919923596498  
,RMSE : 22.607343770546105  
,R2 : 0.529720294909839
```

```
[ ]: df.dtypes
```

```
[26]: .....
```

	0
bhk	int64
type	object
locality	object
area	int64
price	float64
price_unit	object
region	object
status	object
age	object

dtype: object

```
[ ]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

# -----
# Load Dataset
# -----
df = pd.read_csv("/content/house_data.csv")

# -----
# Separate features & target
# (change 'Price' if your target column has a different name)
# -----
X = df.drop("price", axis=1)
y = df["price"]

# -----
# Identify categorical columns
# -----
categorical_cols = X.select_dtypes(include=["object"]).columns
numerical_cols = X.select_dtypes(exclude=["object"]).columns

# -----
# Preprocessing
# -----
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
        ("num", "passthrough", numerical_cols)
    ]
)

# -----
# Model pipeline
# -----
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

# -----
# Train-test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)

# -----
# Train model
# -----
model.fit(X_train, y_train)

# -----
# Predict
# -----
y_pred = model.predict(X_test)

# -----
# Evaluation
# -----
print("---- Model Performance ----")
print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :", mean_squared_error(y_test, y_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2  :", r2_score(y_test, y_pred))

---- Model Performance ----
,MAE : 5.45981251631392
,MSE : 61.00622196018831
,RMSE : 7.810647985934862
,R2  : 0.9438653148533123

```

```
[ ]: print("R2 :", r2_score(y_test, y_pred))
```

```
R2 : 0.9438653148533123
```

```
[ ]: import joblib
```

```

# Save model
joblib.dump(model, "house_price_model.pkl")

```

```
print("Model saved successfully!")
```

```
Model saved successfully!
```